



Department of Computer and Software Engineering
College of E&ME, NUST, Rawalpindi



CS114: Fundamentals of Programming

Lab Project:

Course Instructor:

Asst. Professor Jahan Zeb

Lab Instructor:

Ma'am Ayesha Khanum

<u>Sr. No.</u>	<u>Name</u>	<u>CMS ID</u>
1	Taimoor Atif	548296
2	Hadi Khan Lodhi	541238
3	Muhammad Bilal Nadeem	542808
4	Ahmad Shafi	518046
5	Haris Talha	578468



Department of Computer and Software Engineering
College of E&ME, NUST, Rawalpindi



Software Requirement(s): VS code, Command Prompt

Hardware Requirement(s): Laptop

Intro:

Welcome to Ashen Realms, a console-based RPG where every step across the map can change your fate. You play as a wandering hero navigating villages, castles, forests, ruins, and enemy camps. Unlike a simple combat-only adventure, this world reacts to your political choices. Helping villagers, negotiating with nobles, or taking secret missions can raise your reputation or make you a target. As suspicion grows, guards begin watching, and one wrong move can end your journey. Combat encounters test your strength, while exploration rewards risk-taking with gold, items, and experience. Your goal is to survive, grow stronger, and earn the kingdom's trust before it turns against you.

Game Info / Parameters:

- Core stats & systems: Health/MaxHealth, Gold, Level/XP progression, Attack/Defense, plus an Inventory system (potions, weapons, artifacts) that directly affects survival in combat and exploration.
- Political mechanics: Two key values Reputation (trust) and Suspicion (danger) control access to restricted/secret events and can trigger a loss condition if suspicion becomes too high.
- World + progression rules: An 8×8 dynamic map generates locations (Village/Castle/Forest/Ruins/Enemy/Empty), turns are counted, events are tracked to prevent repeats, and the game supports saving/loading and a leaderboard scoring system.



Department of Computer and Software Engineering
College of E&ME, NUST, Rawalpindi



Overview:

Ashen Realms: A Console-Based Political RPG Simulation is a C++ console game made from multiple modules. The player moves on an 8×8 randomly generated map and triggers different events depending on the tile: Village (V), Castle (C), Forest (F), Ruins (R), Enemy (E), or Empty (.).

The game is “political” because choices in villages and castles change Reputation and Suspicion, which affects trust, unlocks secret/restricted events, and can lead to losing if suspicion becomes too high. Combat, exploration rewards, saving/loading, and a leaderboard are also included.

Bilal — Player System (player.h, player.cpp):

Bilal’s code is the foundation of the whole project because it defines the Player struct and the functions that update player stats. The Player includes health, gold, reputation, suspicion, level/XP, attack/defense, map position, inventory, and tracking stats like turns played.

- Initialization: initializePlayer() sets starting stats (HP 100, Gold 50, Level 1) and adds starter items (sword, shield, potions).
- Stat updates: functions like updateHealth, updateReputation, updateSuspicion, and addGold/spendGold modify stats and print messages so the player immediately sees what changed.
- Level system: gainExperience and checkLevelUp increase the player’s level, improve stats, and add a “Level Token” to inventory.
- Inventory system: items are stored in a fixed array; addToInventory and removeFromInventory manage item counts and categorize items (potions/weapons/artifacts).



Department of Computer and Software Engineering
College of E&ME, NUST, Rawalpindi



Output: This module produces most of the feedback text such as gold gained/spent messages, reputation updates, warning messages when suspicion is high, level-up announcements, and inventory display.

Ahmad-World-Map-System **(world_map.h, world_map.cpp):**

Ahmad's module creates the world map, handles movement, and prints the map each turn.

- Dynamic map creation: `makeMap()` allocates an 8×8 `char**` grid, and `freeMap()` correctly deletes it at the end.
- Random tile generation: `fillMap()` assigns tiles using random chances (some villages/castles, more empty spaces, etc.).
- Movement: `handleMove()` reads WASD input, checks map boundaries, and updates player coordinates.
- Display: `showMap()` prints the full map and a status bar showing HP, Gold, Reputation, Suspicion, and Level.

Output: Every turn the player sees the map grid with a legend and their current stats; invalid moves print an error message.

Hadi — Combat System (combat.h, combat.cpp):

Hadi's code runs the turn-based combat when the player encounters enemies in forests or enemy camps.

- Enemy generation: `generateEnemy(playerLevel)` randomly picks an enemy type (Bandit, Wolf, Mercenary, Corrupt Knight) and scales enemy stats with player level.
- Damage logic: `calculateDamage()` ensures damage is always at least 1 and reduces damage based on defense.



Department of Computer and Software Engineering
College of E&ME, NUST, Rawalpindi



- Combat loop: startCombat() displays the encounter, then repeats turns until either the enemy or player is defeated. The player can Attack, Defend, Flee, or Use Potion.
- Rewards: defeating enemies gives random gold, reputation, and XP and increases enemies defeated count.

Output: Combat shows a menu each round, prints damage results, shows flee success/failure, and prints rewards when the enemy is defeated.

Taimoor — Politics System (politics.h, politics.cpp):

Taimoor's module provides the “political RPG” part through story events and choices.

- Political events: Each event has a title, story text, 3 choices, and stat effects (reputation/suspicion/gold).

Event restrictions:

- isRestricted events require rep > 40 and suspicion < 30.
- isSecretMission events require rep > 70 and suspicion < 20.
- No-repeat system: PoliticsState stores event IDs that already happened using logEvent() and checks them using hasEventFired().
- Triggering events: runVillageEvent() and runCastleEvent() select the next unfired event and call triggerEvent() to display it and apply the player's decision.

Output: The player sees a “POLITICAL EVENT” box with story text and 3 options, then sees updated reputation/suspicion/gold messages plus a political standing summary.

Haris — Save System & Leaderboard (save system.h, save system.cpp):



Department of Computer and Software Engineering
College of E&ME, NUST, Rawalpindi



Haris's code makes the game persistent and competitive through saving/loading and score ranking.

- Saving: `saveGame()` writes all player stats and inventory into `save.txt`.
- Loading: `loadGame()` reads the same format back into the Player and calls `clampStats()` to keep values valid.

Leaderboard: scores are computed with

$$\text{score} = (\text{level} \times 100) + (\text{reputation} \times 10) + \text{gold} - (\text{suspicion} \times 5)$$

The top 5 scores are stored in `leaderboard.txt`, sorted, and displayed.

Output: The player sees confirmation messages for saving/loading and a formatted leaderboard table showing rank, name, level, gold, reputation, and score.

Main Program Flow — Driver (Dext.cpp):

Dext.cpp is the control center that connects all modules.

- Shows main menu (New / Load / Leaderboard / Quit).
- Creates or loads the player, then initializes politics state.
- Builds the map and ensures the starting tile is empty.
- Runs the main loop: show map → get input → move → increase turn counter → check tile and trigger an event.
- Win condition used in main: $\text{Reputation} \geq 80$ and $\text{Turns Played} \geq 20$.
- Lose condition: $\text{Suspicion} \geq 100$ or HP reaches 0 in combat.
- At the end, the score is saved to the leaderboard and the leaderboard is displayed.

What the console output will show:

- Menu screen at start



Department of Computer and Software Engineering
College of E&ME, NUST, Rawalpindi



- Map + stats bar every turn
- Inventory screen when pressing I
- Save message when pressing X
- Political event prompts when stepping on Village/Castle tiles
- Combat screens in forests/enemy camps
- Random ruins messages (gold, trap damage, artifact)
- Win/Game Over text and then a leaderboard table at the end

Conclusion:

This project showed me how C++ fundamentals come together in a complete game system.

- I used structs and header files to organize shared data like the Player, enemies, and political events across modules.
- I applied functions, conditionals, and loops to build gameplay logic like combat turns, event gating, and win/lose conditions.
- Working with file I/O helped me implement saving/loading and a leaderboard that persists between sessions.
- I also practiced dynamic memory with the 2D map allocation and proper cleanup, which improved my understanding of pointers and memory management.